Training Register Demo - High-Level Design

SYN-TRN-HLD-001 | 1.0.0 | SYNTHETIC TEST FIXTURE

Repository: https://github.com/rdhepe/testrepo

Purpose and ownership

Synthetic input for UC3 assessment scenarios, including UC3-WF-01, TC-DS-01, TC-DS-03, TC-DS-05 and TC-DS-06. This is not a PETRONAS production application, approved enterprise design, or compliance certification.

Training Register Demo (SYN-TRN-001), version 1.0.0, registers employees for training and tracks course completion. Business owner: Synthetic Learning Operations. Technical owner: Synthetic Demo Engineering. Contact: training.owner@example.test.

Components, deployment and data flow

Component COMP-HTTP-01: A local browser or HTTP client calls one Node.js HTTP service implemented in src/server.js. GET / serves public/index.html and GET /app.js serves public/app.js. The registration frontend lists and submits enrolments through /api/enrolments. The service binds to 127.0.0.1, port 3100 by default; PORT can override the port. No cloud resource, identity provider, or external integration is specified.

Component COMP-DB-01: The application persists enrolment records in a PostgreSQL database. POST /api/enrolments writes each accepted enrolment to PostgreSQL, and GET /api/enrolments reads the stored records. Accepted enrolments remain available after the Node.js service restarts.

Data flow: local HTTP client -> Node.js HTTP service -> PostgreSQL enrolment database -> JSON response to client. fixtures/application.json supplies application metadata and two synthetic seed records. The separate TC-DS-03 key template is an undeployed configuration fixture, not a database or service deployment. No deployed resource inventory is supplied; this document alone does not verify deployed state.

API and data contract

GET /health returns health and application ID. GET /api/application returns metadata and declared data fields. GET /api/enrolments lists records. POST /api/enrolments validates and adds a record, returning HTTP 201 and a generated id UUID. Seed records have no generated id.

POST accepts only employeeId, employeeName, workEmail, courseCode and completionStatus. All are required non-empty strings with a maximum length of 160 characters. workEmail must have email syntax. completionStatus is enrolled or completed. Unknown fields are rejected. The request size limit is 16 KiB.

Invalid input returns HTTP 400, unsupported content type 415, oversized input 413, and unknown routes 404. No passwords, national IDs, IP addresses, or authentication tokens are stored in enrolment records.

Trust boundary and limitations

The local machine is the deployment boundary. Loopback binding limits network exposure but does not authenticate local clients. HTTP is not TLS protected. There is no authentication, authorization, MFA, audit trail, encryption-at-rest mechanism, automated retention, backup or recovery.

Use synthetic records only. Do not expose the service publicly or store real personal data. These limitations are intentional and are not claims of passing other UC3 privacy or security controls.

TC-DS-03 key storage and rotation requirement

Control TC-DS-03: The proposed enrolment encryption key shall be stored in a managed key service with access restricted to distinct key-administrator and application roles. Automatic key rotation shall be configured at least every 90 days. The interval is a synthetic assessment requirement, not an asserted enterprise standard.

The configuration under review is Resources.EnrolmentKey in infra/tc-ds-03-key.json, an AWS CloudFormation AWS::KMS::Key resource with AWS_KMS origin and a symmetric encryption key. The key policy restricts administration and cryptographic usage to separate role parameters. In a KMS key policy, Resource '*' refers to the key to which the policy is attached; it is not a public principal. No key material, credentials or actual account identifiers are committed.

The template deliberately omits EnableKeyRotation and RotationPeriodInDays. Permission to call

kms:EnableKeyRotation is not an enabled rotation schedule; an expiry or deletion setting is not rotation either. Expect restricted managed storage configuration to be recognized and the missing required automatic rotation configuration to be reported as Non-Compliant.

This template is an undeployed assessment fixture and is not connected to src/server.js. The application still stores records in memory without encryption. Evaluate the key configuration separately from runtime encryption; do not claim that employee records are encrypted, a key exists in a live account, or live permissions and rotation have been verified. Do not deploy the template.

TC-DS-05 identity-provider and MFA requirement

Control TC-DS-05: Training Register Demo is classified as an internal employee application for this synthetic assessment. Employee enrolment records are designated protected sensitive data for this control, not a claim that they are special-category personal data. Access to GET and POST /api/enrolments shall require authentication through Microsoft Entra with MFA before records can be read or created.

The supplied local application deliberately has no identity-provider integration, token validation or authentication gate on these routes. Anonymous GET returns records with HTTP 200; anonymous valid POST returns HTTP 201; a request bearing an invalid token can still read records. Loopback binding is not authentication. Expect a Non-Compliant finding for access without the required approved identity provider.

The worksheet assumes MFA is enabled when an identity provider is used. Record that as an assessment assumption, not proof of actual enforcement. Here no provider protects the routes, so that assumption cannot satisfy the control. Repository review cannot perform login challenges or establish effective tenant MFA policy. Do not claim that a real tenant has MFA disabled. External-application provider alternatives are outside this internal fixture's scope.

TC-DS-06 sensitive-data audit logging requirement

Control TC-DS-06: Reads through GET /api/enrolments and creation through POST /api/enrolments shall produce audit events containing authenticated user identity, UTC timestamp, action, record reference and outcome. Enrolment records are designated protected sensitive data for this synthetic control, not special-category personal data.

A list-read event shall identify the enrolment collection and returned record references; a create event shall identify the generated record id. Seed records have no id, so their audit references shall be pseudonymous stable identifiers rather than full personal-data payloads. Record the attempt before access or mutation and its outcome when known, linked by a correlation identifier. Audit events shall not contain full request or response payloads, learner names, email addresses, credentials or tokens.

The existing GET handler directly returns enrolments, and the POST handler calls enrolments.push(record) and returns HTTP 201 without an audit event or audit sink. The startup console.log reports only the listening URL and is not an audit trail. The expected verdict is Non-Compliant for missing read/create audit logging. Authentication is also absent as tracked separately by TC-DS-05; request-supplied identity is not authenticated identity.

Only read and create are implemented; update and delete are outside this fixture's scope. Review src/server.js and its dependencies for logging calls and their control-flow relationship to the operations. A generic console message or a pre-operation message alone does not establish a complete audit event or recorded outcome. GitHub-only analysis cannot establish live log delivery, retention, tamper protection or external infrastructure logging.